

A Hybrid Rasterized Ray- Traced 3D Graphics Renderer

...

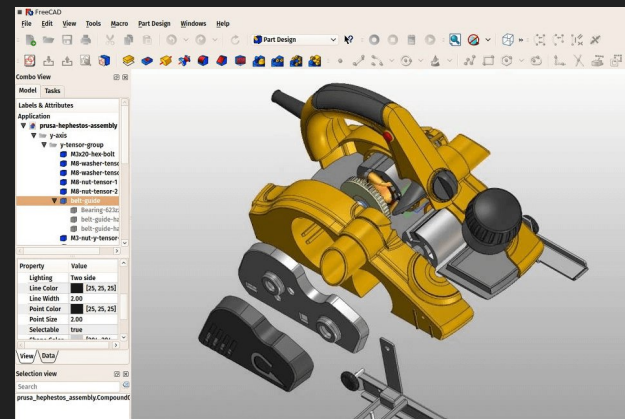
Toby Hothersall

Introduction

What is 3D rendering

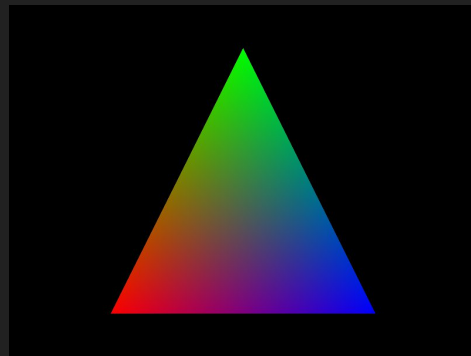
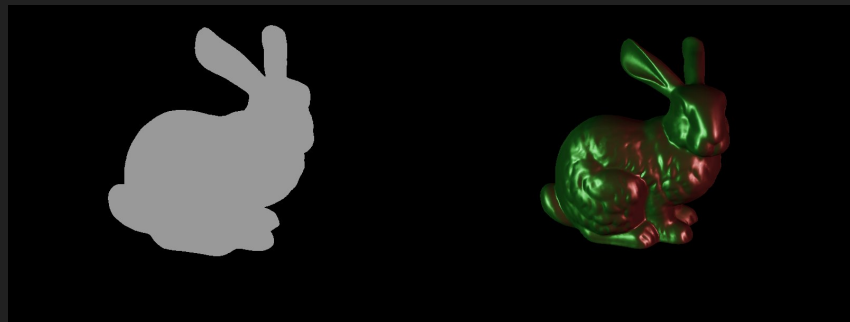
Projecting 3D scenes onto a 2D screen; a scene is composed of one or more meshes which are composed of a set of geometric primitives.

- CAD
- CGI in films
- Video games



What is this project

- A hybrid rasterized ray traced 3D graphics renderer.
- Single thread, CPU based.
- Is a modern processor powerful enough for useful rendering?
- In what scenarios would you use a raster pipeline or a ray tracing pipeline?
- What is the difference between raster images and ray traced images in terms of rendering speed and quality?



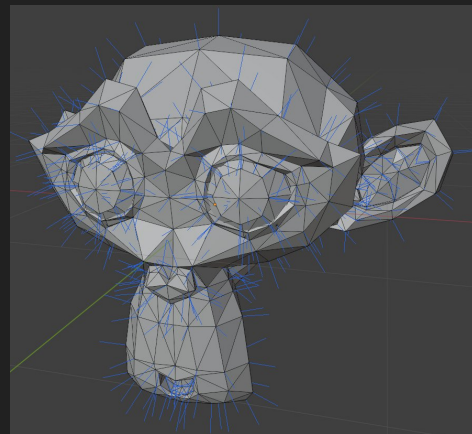
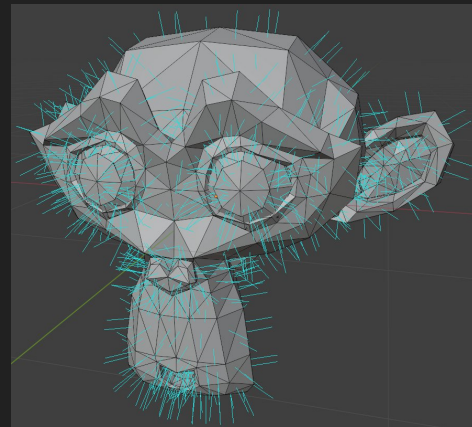
Background

What is a Mesh

- Mesh -> Face -> Vertices
 - Quads vs triangles
- Vertex: position, colour, normal, UV coordinates
 - Normals per vertex or per face?
- Vertex and index buffers

```
struct Vertex {  
    Vector position;  
    Colour colour;  
    Vector normal;  
    double u;  
    double v;  
} typedef Vertex;
```

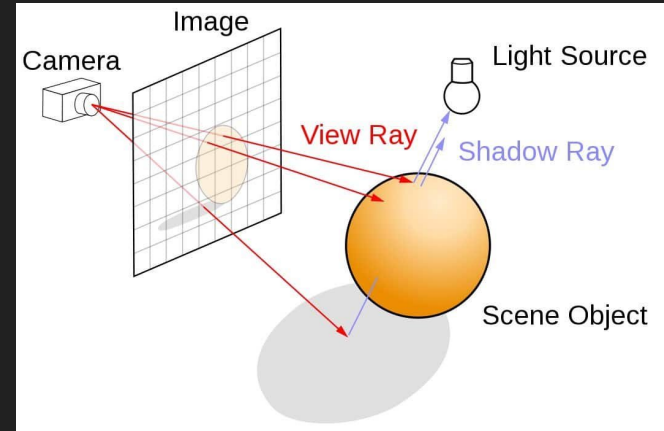
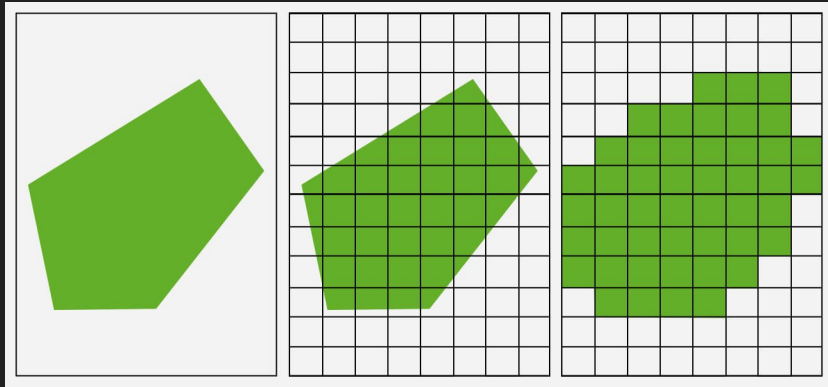
```
struct Index {  
    size_t position;  
    size_t colour;  
    size_t uv;  
    size_t normal;  
} typedef Index;
```



Rasterization vs Ray Tracing

Rasterization Projects meshes onto the screen via a series of transformations.

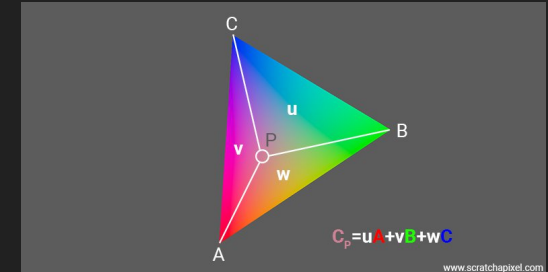
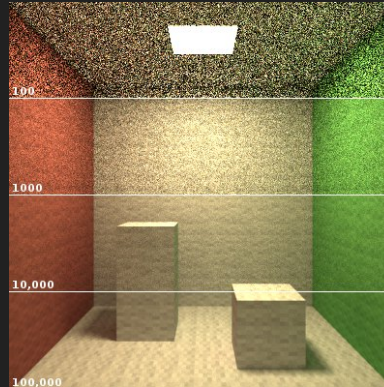
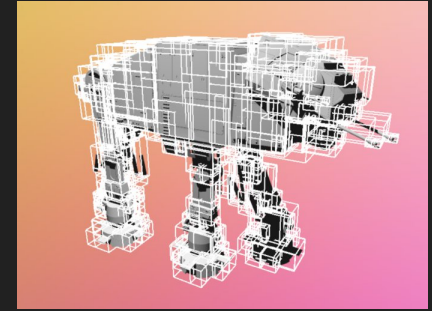
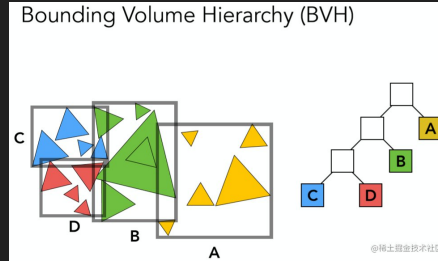
Ray Tracing generates a ray per pixel, then determines the colour of the ray via ray-scene intersection.



Design and Implementation

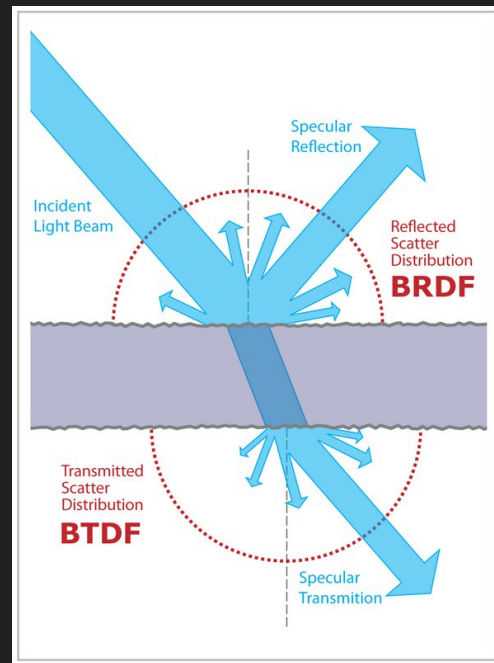
Ray Tracing

- Ray-Triangle intersection
 - Ray equation
 - Moller-Trumbore
 - Barycentric Coordinates
- Acceleration Data Structures
 - Bounding Volume Hierarchies
 - Ray-aabb intersection
- Types of ray tracing
 - Ray casting
 - Recursive ray tracing
 - Path tracing
 - Bidirectional path tracing



The Rendering equation & BxDFs

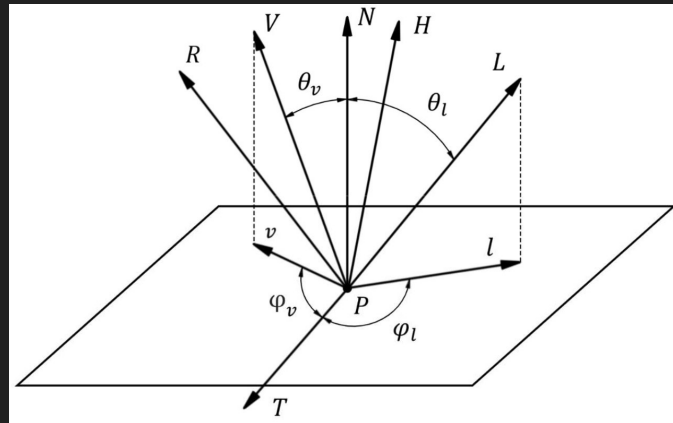
- The Rendering equation
 - Outgoing Light = Incoming light + Reflected light
 - x represents the point hit
 - w_i represents the incoming light vector
 - w_o represents the outgoing light vector
 - w_n represents the normal vector of the surface hit
- Bidirectional Scattering Distribution function
- Lambert's Law



$$L_o(x, w_o) = L_e(x, w_o) + \int_{\Omega} f_r(x, w_i, w_o, w_n) L_i(x, w_i) \cos\theta_i dw_i$$

The Rendering equation & BxDFs

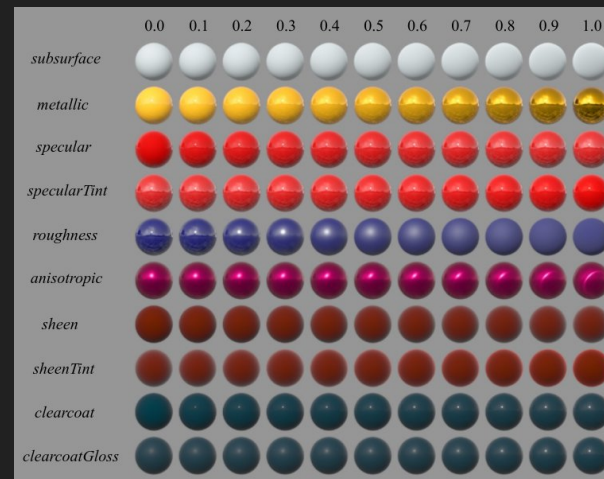
- The Rendering equation
 - Integral becomes a summation
- Bidirectional Scattering Distribution function
 - Takes in vectors relating to the hit point
 - Takes in parameters relating to the material implemented
- Lambert's Law
 - Calculated using the dot product between the normal and light vector



```
struct evaluateBxDF {  
    const Vector& in;           // or light or L  
    const Vector& out;          // or camera or V  
    const Vector& normal;       // or N  
    const Vector& X;            // orthoganol basis vectors  
    const Vector& Y;            // orthoganol basis vectors  
    const Colour& baseColour;   // colour of the material  
  
    Colour operator()(const DisneyBSDF& material) const;  
    Colour operator()(const DisneyDiffuse& material) const;  
};
```

The Rendering equation & BxDFs

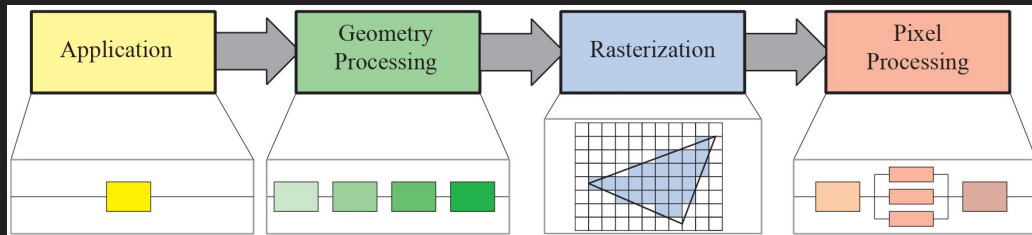
- The Rendering equation
 - Integral becomes a summation.
- Bidirectional Scattering Distribution function
 - Takes in vectors relating to the hit point
 - Takes in parameters relating to the material implemented
 - Allows you to change the properties of the material without changing the underlying algorithm
- Lambert's Law
 - Calculated using the dot product between the normal and light vector



Rasterization

The View Frustum

- Culling and occlusion



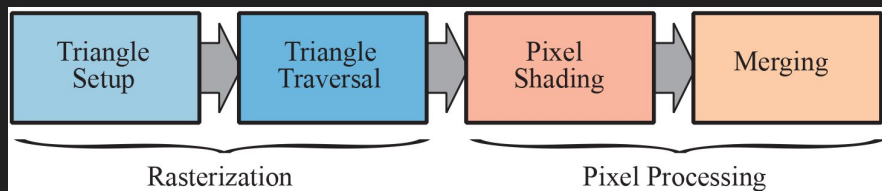
The Rendering Pipeline

- Logical pipeline vs practical pipeline
- The framebuffer and z-buffer



The Geometry Processing Pipeline

- Normalised Device Coordinates
- Canonical View Volume



Rasterization

The View Frustum

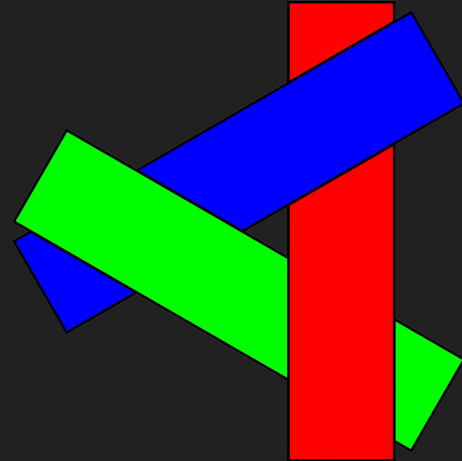
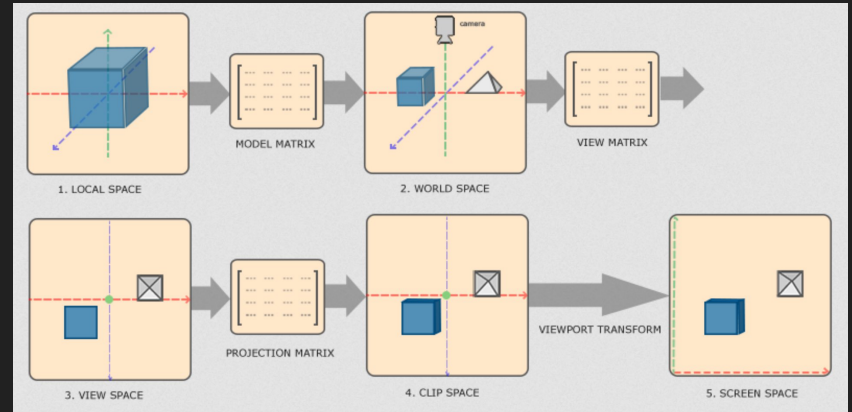
- Culling and occlusion

The Rendering Pipeline

- Logical pipeline vs practical pipeline
- The framebuffer and z-buffer

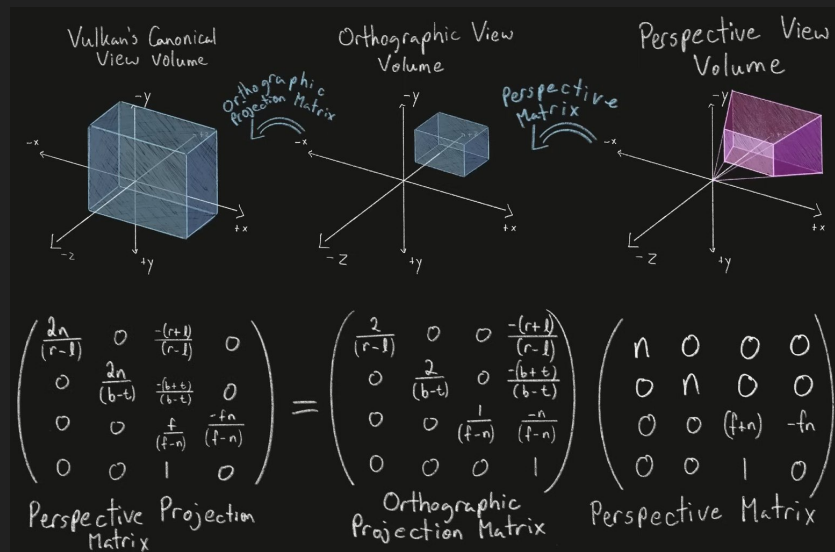
The Geometry Processing Pipeline

- Normalised Device Coordinates
- Canonical View Volume



Transformations

- Affine transformations
 - Scaling
 - Translation
 - Rotation
- Camera Transformation
 - Change of basis followed by a translation
- Projections
 - Orthographics vs Perspective
 - Maths tricks
 - Field of view



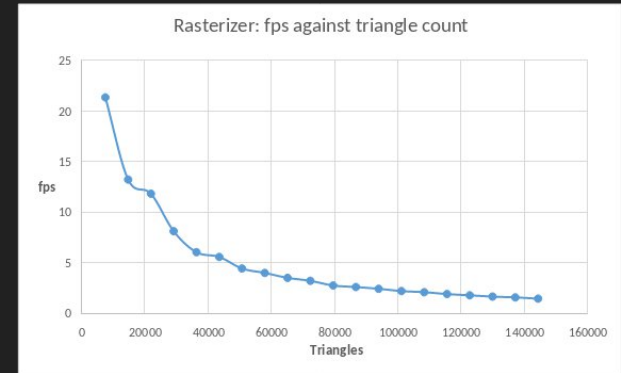
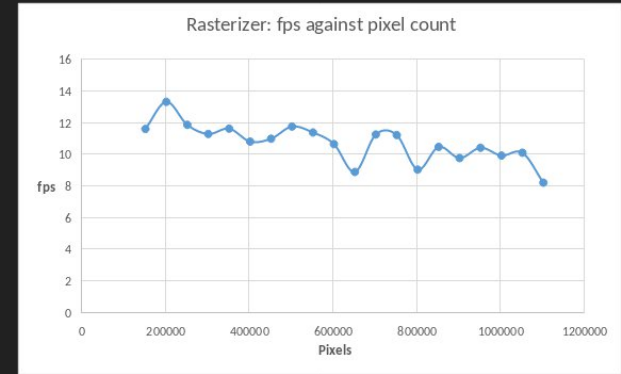
$$P_{(n,f,t,r)} = \begin{bmatrix} \frac{1}{a \cdot \tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\theta/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{f-n} & -2 \frac{f \cdot n}{f-n} \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Testing and Evaluation

Rasterization results

- Offline Rendering?
- Real-Time Rendering?
- Interactive Real-Time Rendering?
- Render Quality?

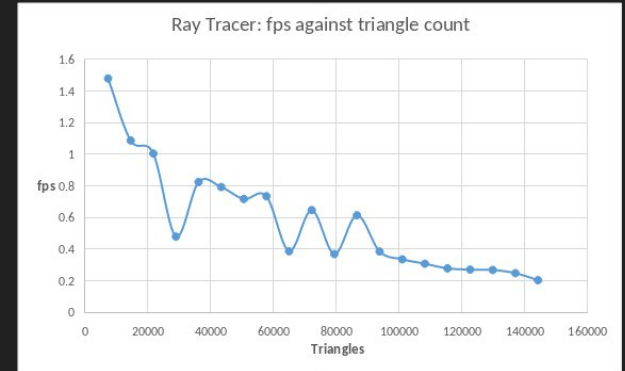
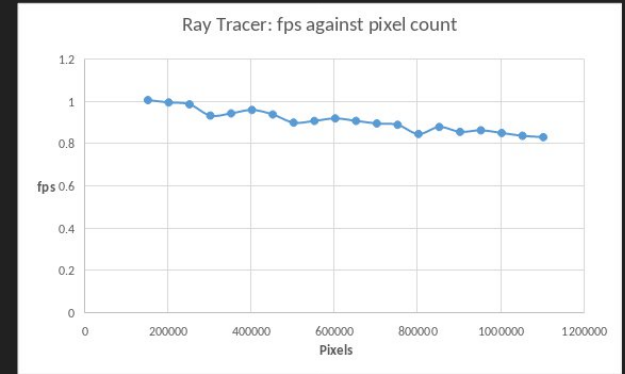
Rasterization Framerate Data:					
		Triangle count:			
		7202	14404	21606	144046
Resolution:	1920x1080	17.659	11.670	8.915	1.521
	1280x720	24.267	14.620	9.840	1.570
	848x480	30.511	15.823	10.855	1.582



Ray Tracing results

- Offline Rendering?
- Real-Time Rendering?
- Interactive Real-Time Rendering?
- Render Quality?

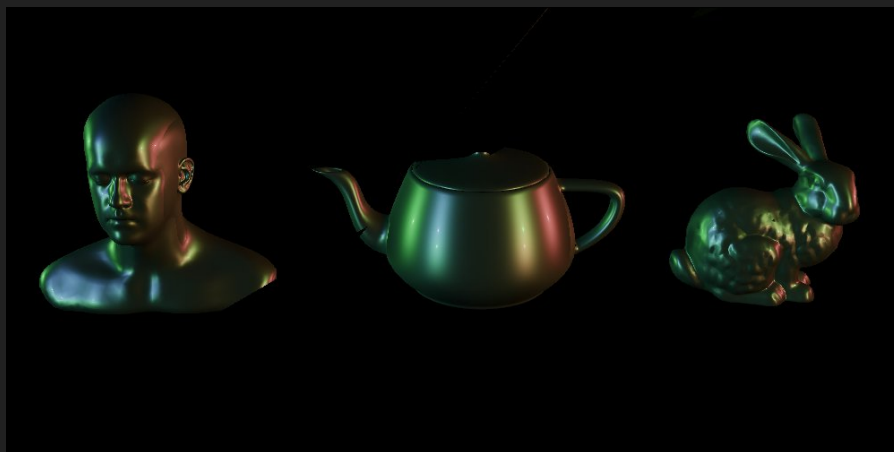
Ray Tracing Framerate Data:					
		Triangle count:			
		7202	14404	21606	144046
Resolution:	1920x1080	0.914	0.769	0.710	0.027
	1280x720	1.189	0.949	0.866	0.055
	848x480	1.375	1.061	0.959	0.109



Conclusions

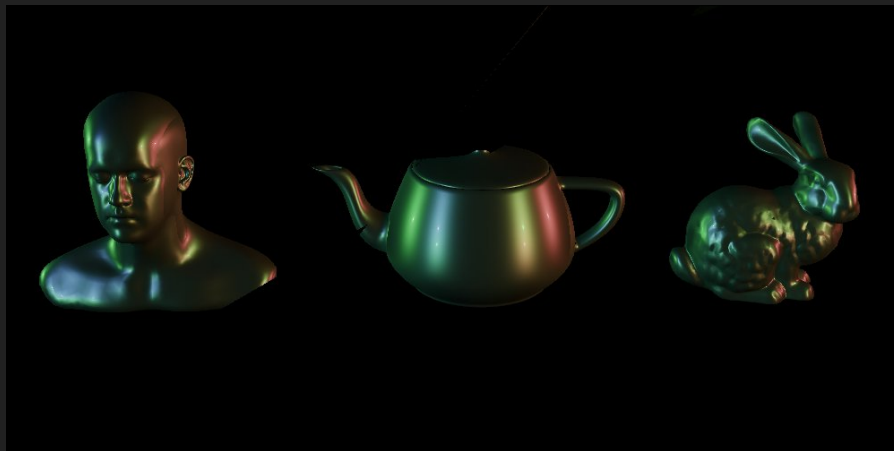
Conclusions

- Is a modern processor powerful enough for useful rendering?
- In what scenarios would you use a raster pipeline or a ray tracing pipeline?
- What is the difference between raster images and ray traced images in terms of rendering speed and quality?



Conclusions

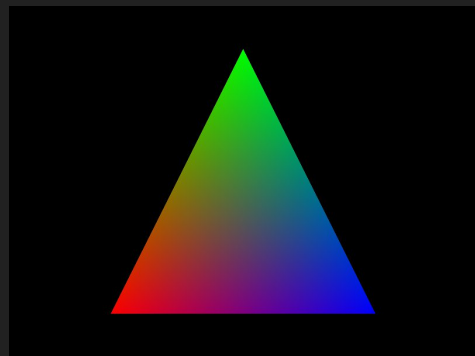
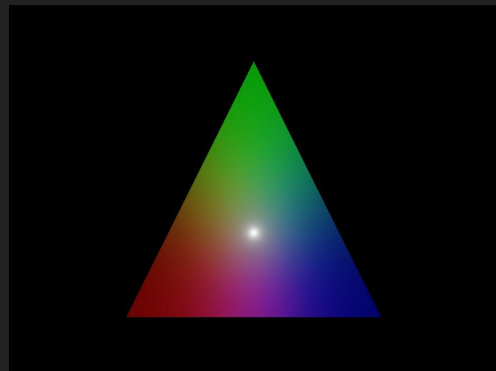
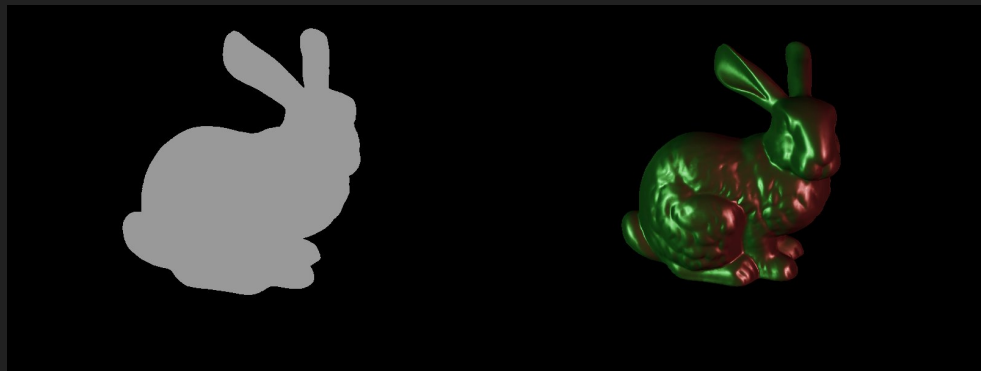
- Is a modern processor powerful enough for useful rendering?
 - Yes
- In what scenarios would you use a raster pipeline or a ray tracing pipeline?
- What is the difference between raster images and ray traced images in terms of rendering speed and quality?



Conclusions

- Is a modern processor powerful enough for useful rendering?
 - Yes
- In what scenarios would you use a raster pipeline or a ray tracing pipeline?
 - Offline rendering - Ray Tracing with >100,000 triangles @ 1080p
 - Real-Time rendering - Rasterization with ~14000 triangles @ 1080p for 24fps or ~7000 triangles @ 720p for 12fps
 - Interactive Real-Time rendering - Rasterization with ~7000 triangles @ 848p
- What is the difference between raster images and ray traced images in terms of rendering speed and quality?

Raster vs Ray Tracer Render Quality



Demo

Questions?